JETZ SPORTS ACADEMY

Attendance Module REST API & Postman Collection Reference

System: JETZ ERP Backend • **Version:** 1.0 • **Auth:** JWT Bearer • **Date:** October 2026

1. API Overview & Quick Reference

The Attendance module powers attendance tracking for sports academy sessions, supporting daily coach roll-calls, mobile-optimized roster fast-paths, atomic bulk recording, group photo AI-assisted presence suggestions, and an immutable correction audit trail. All endpoints strictly validate roster membership, database uniqueness (session, student), and granular server-side RBAC permissions.

Method	Endpoint	Permission	Description
GET	/api/attendance/	attendance.view	List / filter attendance records
POST	/api/attendance/	attendance.mark	Create single record for enrolled student
PATCH	/api/attendance/{id}/	attendance.correct	Correct status (requires reason & audit)
GET	/api/attendance/roster/	attendance.view	Mobile roster lookup with status & photos
POST	/api/attendance/bulk_mark/	attendance.mark	Bulk mark entire session atomically
POST	/api/attendance/photo_analyze/	attendance.mark	Upload group photo for AI face detection
GET	/api/attendance/corrections/	attendance.view	Audit log of all attendance corrections

2. Authentication & Postman Environment Setup

All requests must include the JWT bearer token in the Authorization header: Authorization: Bearer <token>.

Authentication Flow (POST /api/auth/login/)

```
Request URL: {{baseUrl}}/auth/login/
Payload:
{ "username": "admin", "password": "Jetz@123" }
Postman Test Script (Automatic Token Capture):
var json = pm.response.json();
if (json.access) { pm.collectionVariables.set('token', json.access); }
```

Postman Collection Variables:

- **baseUrl**: http://localhost:8000/api (Backend server API root)
- **token**: Auto-populated JWT Access Token from Login
- **sessionId**: Primary key of the Session (e.g., 1)
- **studentId**: Primary key of the Student (e.g., 1)
- **recordId**: Primary key of the AttendanceRecord (e.g., 1)

3. Detailed Endpoint Specifications

A. Session Roster Fast-Path

Permission: attendance.view

Path: **GET** /api/attendance/roster/?session={sessionId}

Description: Returns all students assigned to the session along with current attendance state, reference photo status, and who marked them. Used directly by the mobile trainer app upon opening a session.

Parameters:
• session (Query param, required): ID of the session.

Sample Response:

```
{ "session": 1, "roster": [ { "student": 10, "student_id": "STU-1002", "name": "Alex Rivera", "status": "present", "record_id": 42, "photo": "http://localhost:8000/media/students/alex.jpg", "has_reference": true, "marked_by": 3, "marked_by_name": "coach1", "marked_at": "2026-10-22T09:15:00Z", "makeup_for": null } ], "is_makeup": false, "makeup_for": null }
```


B. Bulk Mark Session Attendance

Permission: attendance.mark

Path: **POST** /api/attendance/bulk_mark/

Description: Submits attendance for all students in a session in one atomic transaction. Validates roster membership. Idempotent per (session, student). If changing an existing mark, automatically creates an audit record.

Parameters:
Headers: Content-Type: application/json

Request Body:

```
{ "session": 1, "reason": "Morning batch roll call", "items": [ { "student": 10, "status": "present", "makeup_for": null }, { "student": 11, "status": "absent", "makeup_for": null }, { "student": 12, "status": "late", "makeup_for": null } ] }
```


Sample Response:

```
{ "session": 1, "results": [ { "student": 10, "status": "present", "created": true, "makeup_for": null }, { "student": 11, "status": "absent", "created": true, "makeup_for": null }, { "student": 12, "status": "late", "created": true, "makeup_for": null } ] }
```


C. AI Photo Analyze for Attendance

Permission: attendance.mark

Path: **POST** /api/attendance/photo_analyze/

Description: Uploads a session group photo for AI face detection / presence suggestions. Stored as audit evidence. Does NOT directly write attendance—returns confidence suggestions for the coach to confirm via bulk_mark.

Parameters:
Headers: Content-Type: multipart/form-data
• session (text): Session ID.
• photo (file): Image file (JPEG/PNG/WEBP, min 320x320, max 10MB).

Request Body:
```
Form-data: session=1 photo=@group_session_photo.jpg
```

Sample Response:

```
{ "session": 1, "evidence_id": 8, "persons_detected": 12, "quality": { "brightness": 128.4, "contrast": 45.2, "width": 1920, "height": 1080 }, "suggestions": [ { "student": 10, "suggested": "present", "confidence": 0.94 }, { "student": 11, "suggested": "not_marked", "confidence": 0.20 } ] }
```

D. List & Create Attendance Records

Permission: attendance.view / attendance.mark

Path: GET / POST /api/attendance/

Description: List paginated attendance records with filtering, or create a single attendance record for an enrolled athlete.

Parameters:
• Query params: ?session=, ?student=, ?status=, ?search=, ?page=

Request Body:
```
POST Payload: { "session": 1, "student": 10, "status": "present", "makeup_for": null }
```

Sample Response:
```
{ "id": 42, "session": 1, "student": 10, "student_name": "Alex", "student_ref": "STU-1002", "status": "present", "marked_by": 3, "marked_by_name": "coach1", "marked_at": "2026-10-22T09:15:00Z" }
```

E. Correct Record & Corrections Audit Trail

Permission: attendance.correct / attendance.view

Path: PATCH / GET /api/attendance/{id}/ & /api/attendance/corrections/

Description: Modifying attendance on a locked record requires a mandatory reason string. System creates a corresponding AttendanceCorrection audit entry with old status, new status, and timestamp.

Parameters:
• /api/attendance/corrections/?record=&record;__session=

Request Body:
```
PATCH /api/attendance/42/ { "status": "present", "reason": "Athlete arrived 10m late due to school exam" }
```

Sample Response:
```
Corrections Response: [ { "id": 5, "record": 42, "student_name": "Alex", "old_status": "absent", "new_status": "present", "reason": "Athlete arrived 10m late due to school exam", "corrected_by_name": "manager", "created_at": "2026-10-22T10:05:00Z" } ]
```

4. Key Business Rules & HTTP Status Reference

- **Unique Attendance Constraint:** Database uniqueness guarantees exactly one AttendanceRecord per (session, student).
- **Roster Enforcement:** Students can only be marked if they are in the session's assigned batch roster.
- **Parent In-App Notifications:** Marking a child present fires an automatic in-app notification to linked parent accounts.
- **Locking & Permissions:** Once a session is completed, records are locked. Status mutations require attendance.correct and a non-empty reason.
- **Make-up Session Links:** Make-ups can be associated with missed sessions in the same sport via the makeup_for foreign key.
- **Status Codes:** 200 OK (Success), 201 Created, 400 Bad Request (Validation failure / missing reason), 403 Forbidden (Permission / scope denied), 404 Not Found.